CMSC433 / Part 01

Type foundations

Guided practice

Use with the chat: read the idea and example, then do Question 1 only. We discuss each attempt before continuing.

Idea. Application matches an argument's type with a function's input. Keep lists, pairs, and constraints in your answer. Arrows group to the right; function application groups to the left.

Worked example. In `\h x -> h (Just x)`, let `x :: a`. Then `Just x :: Maybe a`, so `h :: Maybe a -> b`. The whole type is `(Maybe a -> b) -> a -> b`.

1. Match the shapes

Infer `\g x -> g [x]`. Fill the ledger before writing the full type.

Answer: x :: [ ]; [x] :: [ ]; g :: [ ]; result :: [ ]; whole lambda :: [ ].

2. Keep the necessary constraint

Give the most general type of `\xs -> [even x | x <- xs]`. Show which operation introduces a constraint; do not default to `Int`.

Answer: [Type and reasoning.]

3. Diagnose a conflict

Give the most general type or explain why `\f -> (f True, f 'q')` is ill-typed. State the incompatible requirements on `f`.

Answer: [Type or conflicting equations.]

Source: Sample Q1, p. 1; Monads/GenericMonads lecture signatures.